Deep Dive into enabling 1 Million GPUs to talk to Thousands of Smart Devices

Today’s Focus: Partitioning

Pankaj Mehra (mod)

Professor of Research, CSE at Ohio State
CEO at Elephance Memory

Panelists:

Galen Shipman, Los Alamos National Lab

Anand Jayarajan, Nvidia

Evangelos Georganas, Intel

Atharva Chougule, Stanford



OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

Standardizing Commonly Used Partitioning Functions

- Context
  - Standard APIs for AI and Analytics Developers to place Function and Data across 100Ks GPUs and 10Ks of DCC Devices (Computational Memory, Computational Storage, and DPUs)
- Why?
  - Enable Near Data Processing at Scale
  - Increasing portability, performance, and x-vendor interop across Accelerators and DCC Devices
- Representative Issues
  - Compiler-Assisted Partitioning for Offloading Large Operators to Small(er) Devices
  - Aspirational: Automated Generation of Collectives to Gather Partial Results
  - Measuring and Optimizing Workload and Comm Balance across Partitions, and
  - Avoiding x-Partition Comm as much as possible

Panelist Presentations

- **Partitioning Best Practices** from Decades of HPC for Linear Algebra
  - **Galen Shipman**: Widespread use of Hilbert Space Numbering in DOE supercomputers
  - **Evangelos Georganas**: Space Filling Curves for partitioning matrix multiplication on high-core-count servers
- **Open Formats**
  - **Anand Jayarajan**: **D**omain **R**elational **C**alculus as a unifying theme to compile Tensor Ops, Graph Algos, Relational Queries (Substrait) on Arrow data for parallel execution on GPUs
  - **Atharva Chougule**: Compiler-optimized partitioning of Sparse Tensor Functions and Data atop Arrow CSF and others

Additional Questions

- Open Questions
  - Can well-specified partitioning operators eliminate the burden of specifying how partial results should be *gather*ed and *reduce*d?
  - To what extent do we need to investigate partitioning operators on open formats for graphs and tabular data instead of cross-compiling graph algos and analytical queries while also converting data into linear algebra types?
  - Aren't open formats for compute inevitable?
- Additional Operator and Types?
  - Should we look to standardize any Randomized Operators for Parallel DFS Numbering of graphs, multi-objective graph partitioning such as NSGA-II?
  - Extend Arrow open formats for data and compute to include emerging Compressed Embedding and Attention vector types (e.g. in Random Hadamard Rotations of TurboQuant) so that Nearest Neighbor searches occur in quantized domain?

Partitioning for parallelism and offload in HPC

Galen Shipman

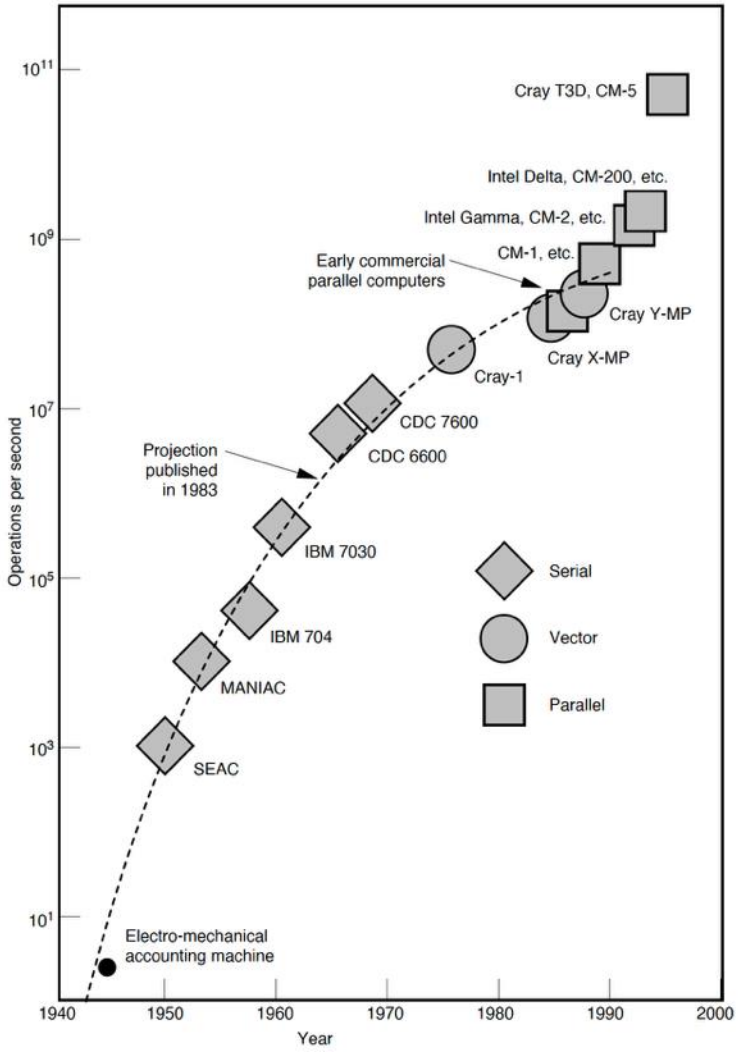
Computer Scientist, Los Alamos National Laboratory



OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

Partitioning for decades



- Since the earliest distributed memory parallel systems such as the Thinking Machines CM-5

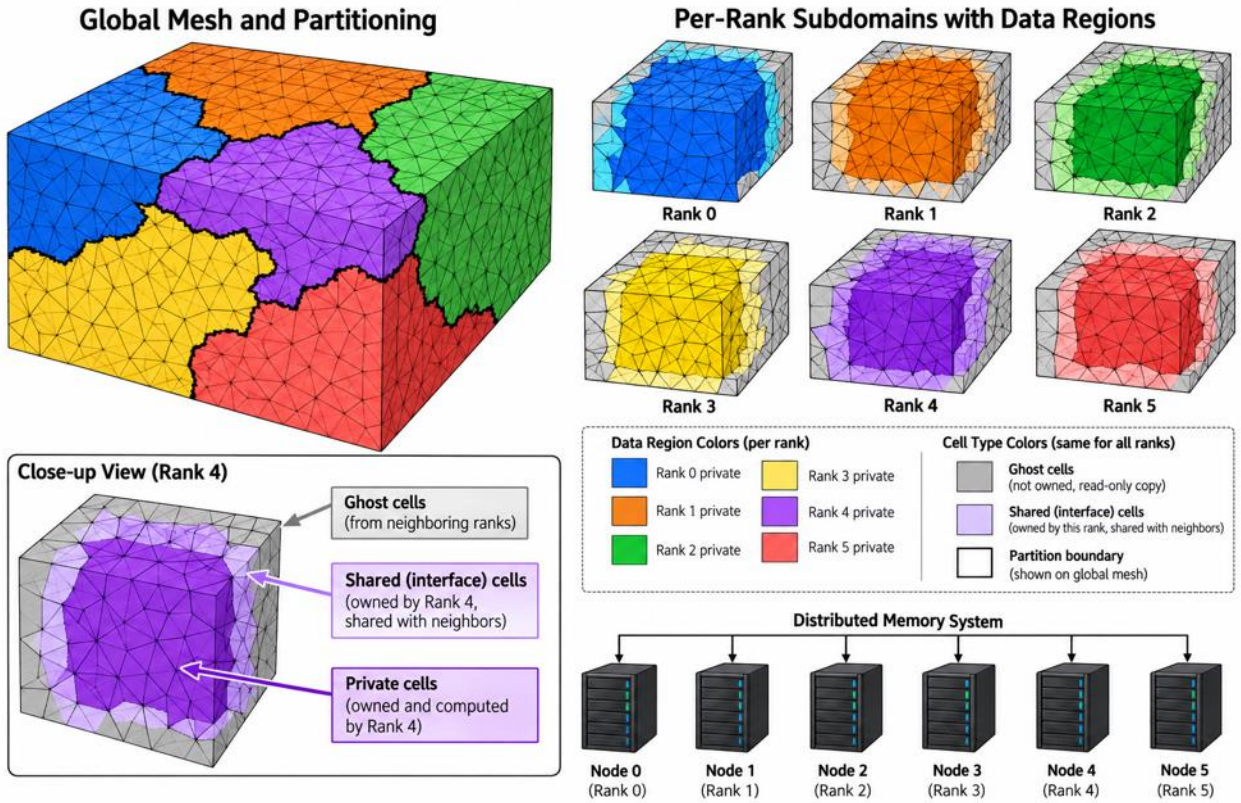


Multiple approaches have evolved

- Static chunk scheduling for fork-join models

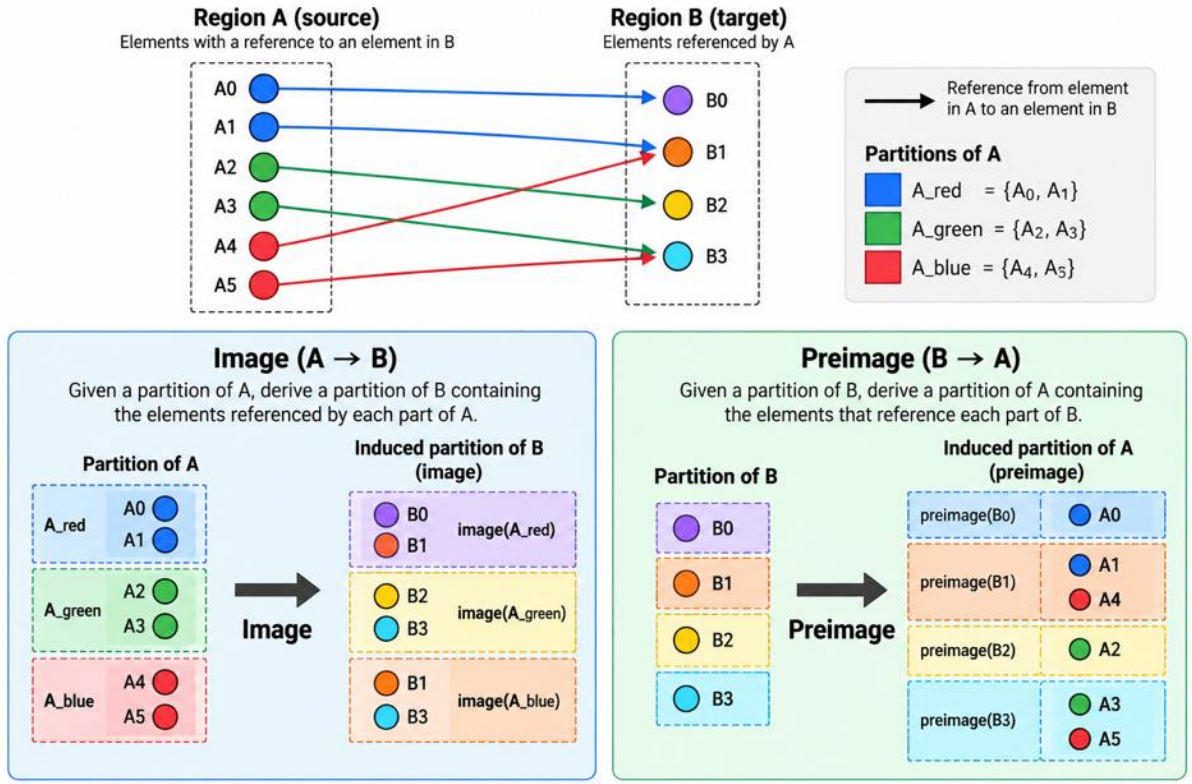
```
#pragma omp parallel for schedule(dynamic, 4)
for (int i = 0; i < 100; i++) {
   // Process data items in parallel with dynamic chunk
partitioning of 4
}
```

- Graph colorings of meshes for SPMD models
  - Adds explicit partitioning of private, shared, and ghost data.



More complexity

- Multiple topological representations of the same underlying geometry
  - Requires transferring data from one topology to another *
- Newer models for SPMD and MPMD
  - Dependent partitioning – A grammar (image, preimage, etc.) and runtime support for powerful partitioning primitives. **



*Plimpton et. al., “Rendezvous algorithms for large-scale modeling and simulation”
**Triechler et. al., “Dependent Partitioning”

Creating the initial independent partition

- Three goals
  - Maximize load balance
  - Minimize ghost/private data ratio – surface to volume
  - Easily compute the initial partition
- Hilbert ordering gives a fast and efficient way to balance these
  - A convenient way to transform a multi-D space into a one-D ordering while preserving locality

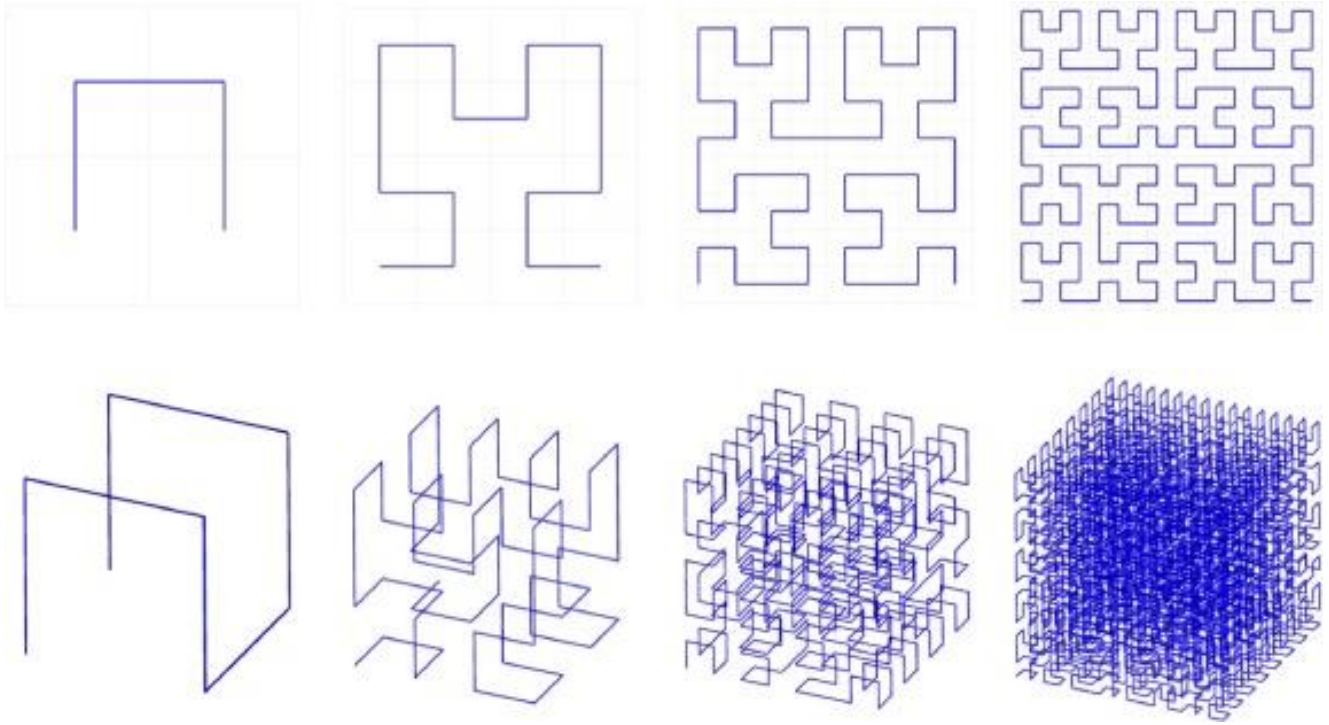


Fig. 1. Hilbert SFC recursive generation, 2D (top) and 3D (bottom).

*Borrell et.al., “Parallel mesh partitioning based on space filling curves”

Space Filling Curves is All You Need : Communication-Avoiding Matrix Multiplication Made Simple

Evangelos Georganas

Intel Corporation

Parallel Computing Lab, CTO Office



OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

https://arxiv.org/abs/2601.16294
https://github.com/libxsmm/sfc_ca_gemm

Legal Disclaimer & Optimization Notice

INFORMATION IN THIS DOCUMENT IS PROVIDED “AS IS”. NO LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS IS GRANTED BY THIS DOCUMENT. INTEL ASSUMES NO LIABILITY WHATSOEVER AND INTEL DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY, RELATING TO THIS INFORMATION INCLUDING LIABILITY OR WARRANTIES RELATING TO FITNESS FOR A PARTICULAR PURPOSE, MERCHANTABILITY, OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT.

Software and workloads used in performance tests may have been optimized for performance only on Intel microprocessors. Performance tests, such as SYSmark and MobileMark, are measured using specific computer systems, components, software, operations and functions. Any change to any of those factors may cause the results to vary. You should consult other information and performance tests to assist you in fully evaluating your contemplated purchases, including the performance of that product when combined with other products.

Copyright © 2015, Intel Corporation. All rights reserved. Intel, Pentium, Xeon, Xeon Phi, Core, VTune, Cilk, and the Intel logo are trademarks of Intel Corporation in the U.S. and other countries.

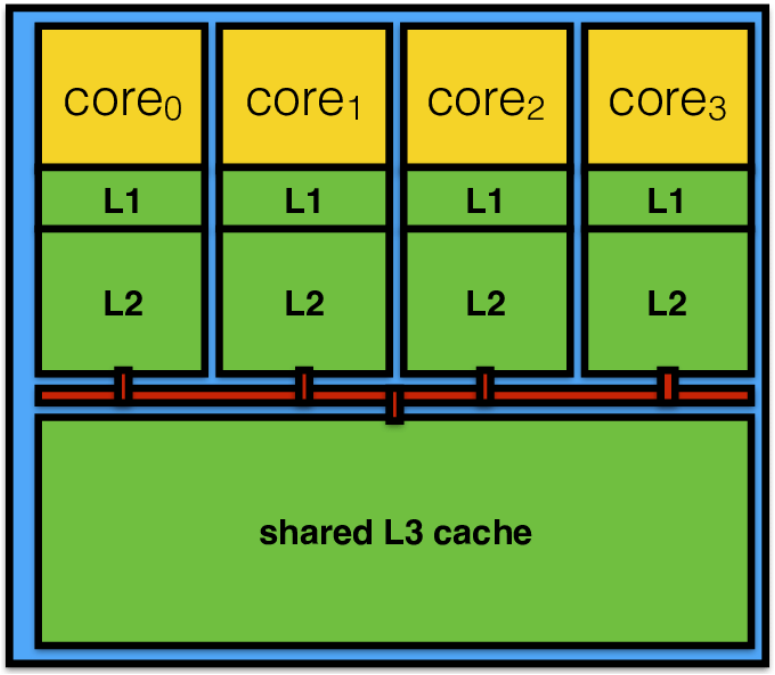
Optimization Notice

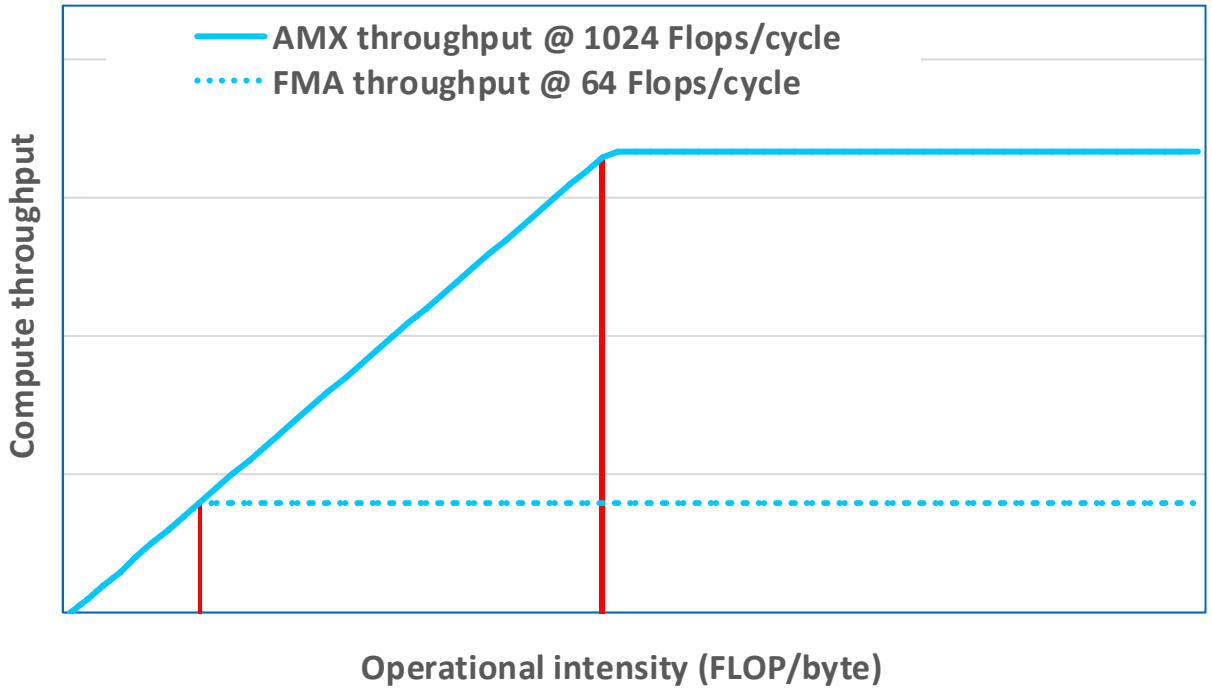
Intel’s compilers may or may not optimize to the same degree for non-Intel microprocessors for optimizations that are not unique to Intel microprocessors. These optimizations include SSE2, SSE3, and SSSE3 instruction sets and other optimizations. Intel does not guarantee the availability, functionality, or effectiveness of any optimization on microprocessors not manufactured by Intel. Microprocessor-dependent optimizations in this product are intended for use with Intel microprocessors. Certain optimizations not specific to Intel microarchitecture are reserved for Intel microprocessors. Please refer to the applicable product User and Reference Guides for more information regarding the specific instruction sets covered by this notice.

Notice revision #20110804

Modern multi-core CPUs are complex…

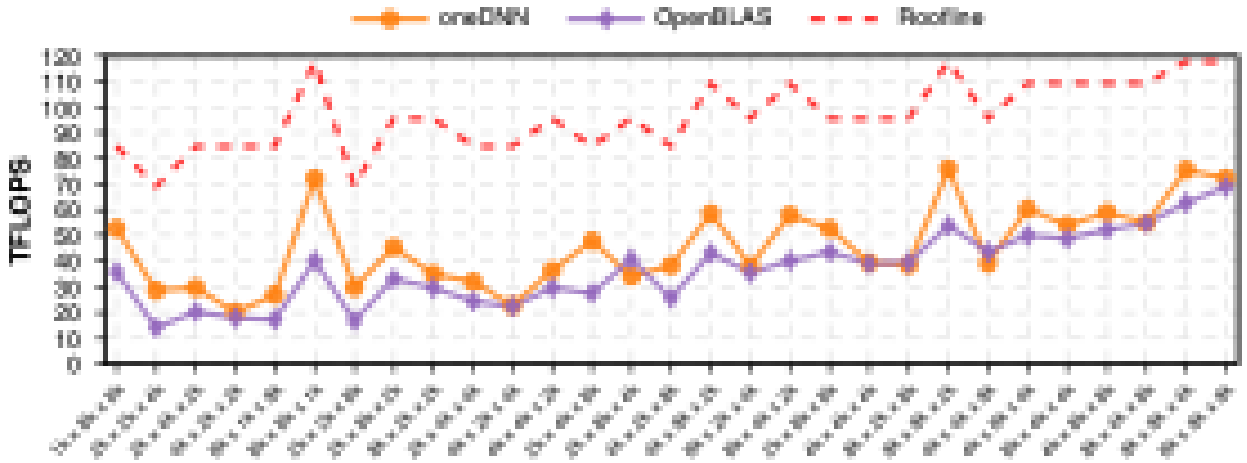
- Modern multi-core CPU systems are increasingly complex
  - Large number of cores
  - Multi-level caches with Last-Level-Cache (LLC) typically shared among cores
  - Matrix Multiplication accelerators (e.g. AMX) skew FLOP/byte machine balance





Why yet another GEMM implementation?

- With respect to GEMM implementation, complex multi-core CPUs mean:
  - Need optimal “single-core” microkernels (e.g. instr. selection, register blocking, prefetching)
  - Need multi-level cache blocking (for L1, L2, LLC)
  - Need optimal parallelization schemes/thread decomposition to exploit shared data reuse
  - Need loop re-orderings to control temporal data reuse
  - **All of the above decisions depend on platform at hand, # of threads and matrix shapes ☹**
- State Of The Art: Design a set of blocking/looping schemes & use heuristics/autotune for platform/# of threads/GEMM shape → **Performance “glass jaws” ☹**



SFC-Based Communication-Avoiding (CA) GEMM

- We revisit the mathematical concept of Space Filling Curves (SFC)
- We use SFC to traverse the *tiled* GEMM iteration space
- We leverage recent advancements in generalized Hilbert (Gilbert) curves
- We get platform-oblivious and shape-oblivious GEMM schemes that exhibit an inherently high degree of data locality
- Extend SFC-based work-partitioning to implement Communication-Avoiding (CA) algorithms that ***provably*** minimize data movement
- **Compact code (~30 C++ LOC), SOTA results on multiple CPU platforms without heuristics/exhaustive auto-tuning**

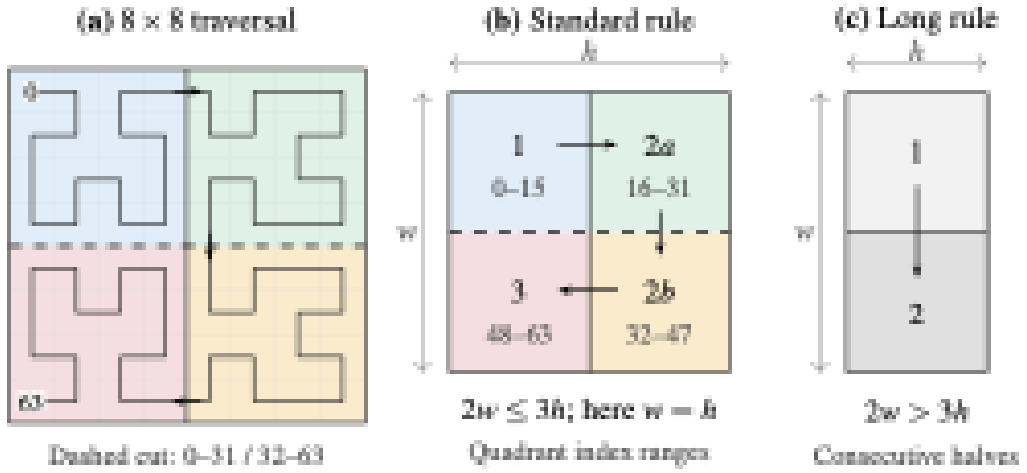
SFC-CA GEMM Algorithm

```
DType A[Mb][Kb][bk/bvnni][bm][bvnni]; //Block M with bm and K with bk
DType B[Nb][Kb][bn][bk]; //Block N with bn and K with bk
DType C[K_layers][Nb][Mb][bn][bm]; //Block N with bn and M with bm

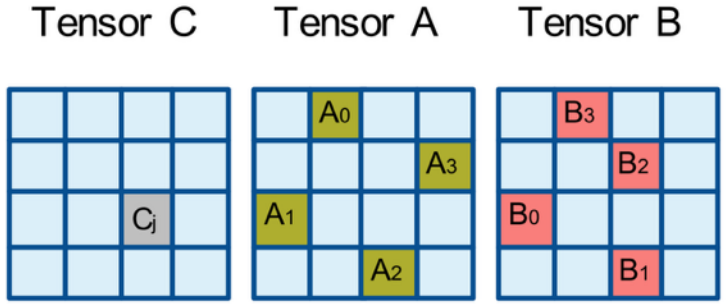
sfc_map = create_sfc_map(Mb, Nb);
Kb_per_layer = Kb / K_layers;
Kb_per_brgemm = Kb_per_layer / k_block_factor;

for (ik = 0; ik < k_block_factor; ik++) {//K-block for A/B panel-size
#pragma omp parallel for
   for (i = 0; i < Mb * Nb * K_layers; i++) {
      i_layer = i / (Mb * Nb); // Index for K_layer
      i_sfc = i % (Mb * Nb);   // ID index for SFC within K-layer
      (im,in) = map_sfc_index(sfc_map,i_sfc);//ID SFC index -> im,in
      //Extract k_block_index based on K-layer (i_layer) & ik K-block
      k_block_index = i_layer * Kb_per_layer + ik * Kb_per_brgemm;
      if (ik == 0) zero_tpp(&C[i_layer][in][im][0][0]);//Zero C block
      // Execute tensor contraction with A/B panels on proper C block
      brgemm_tpp(&A[im][k_block_index][0][0][0],
                 &B[in][k_block_index][0][0],
                 &C[i_layer][in][im][0][0], &Kb_per_brgemm );
   }
}

// Reduce C matrices across K layers
if (K_layers > 1) {
#pragma omp parallel for
   for (i = 0; i < Mb * Nb; i++) {
      in = i % Nb;
      im = i / Nb;
      reduce_stride = M * N;
      C_block_final = &C[0][in][im][0][0];
      add_reduce_tpp(C_block_final, K_layers, reduce_stride);
   }
}
```



- BRGEMM TPP: Batch-Reduce GEMM

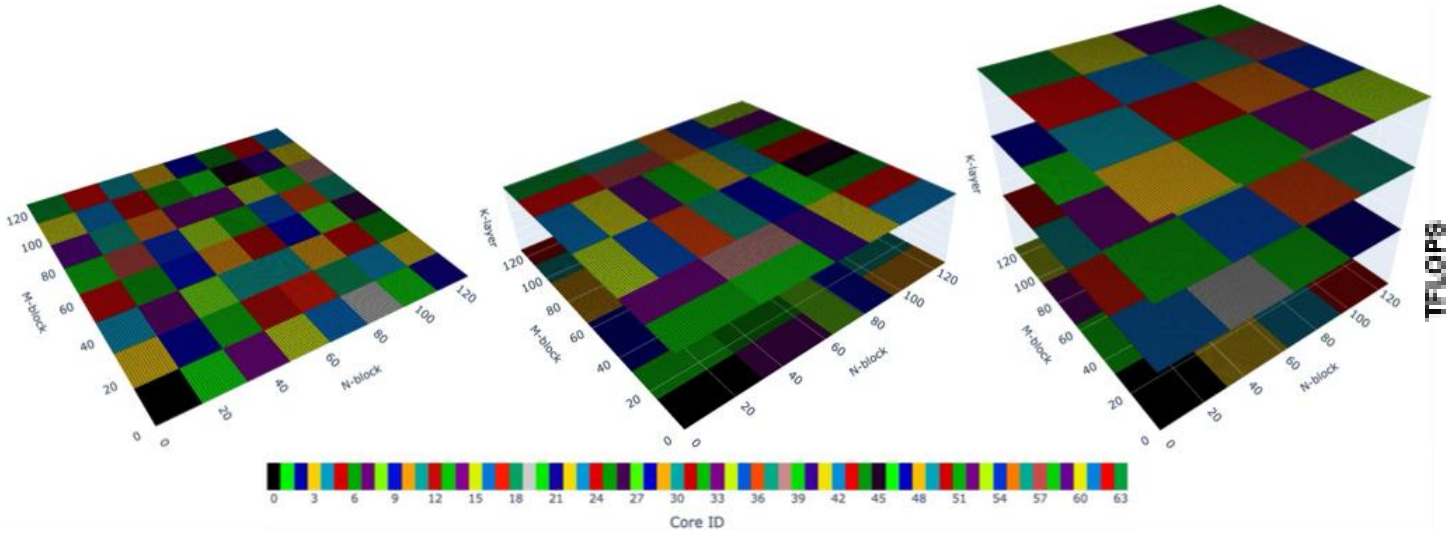


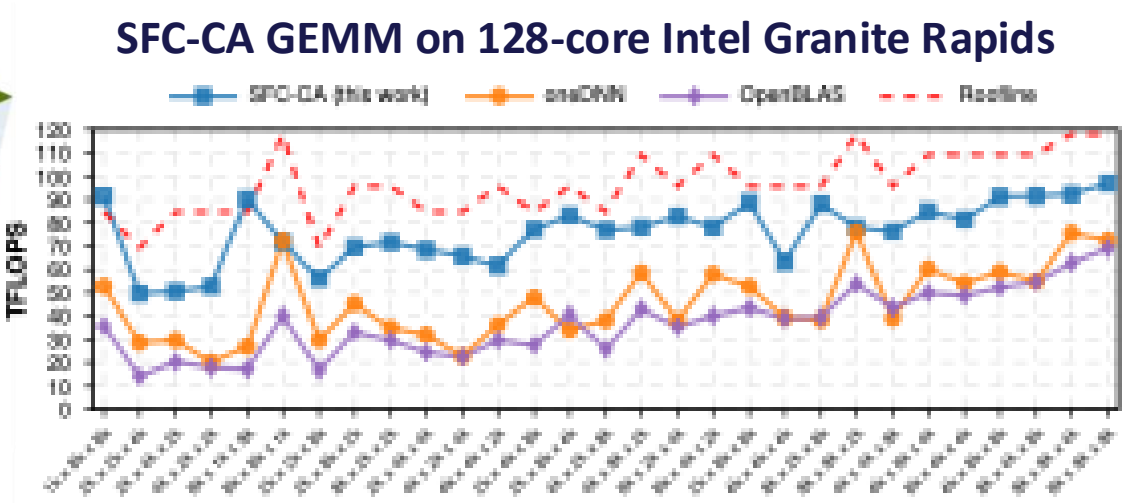
$$C_j = \beta * C_j + \alpha \sum_{i=0}^{N-1} A_i * B_i$$

- Zero TPP: Zeroes out a bn x bm block
- Add_reduce TPP: accumulates bn x bm tiles that are separated by a fixed stride

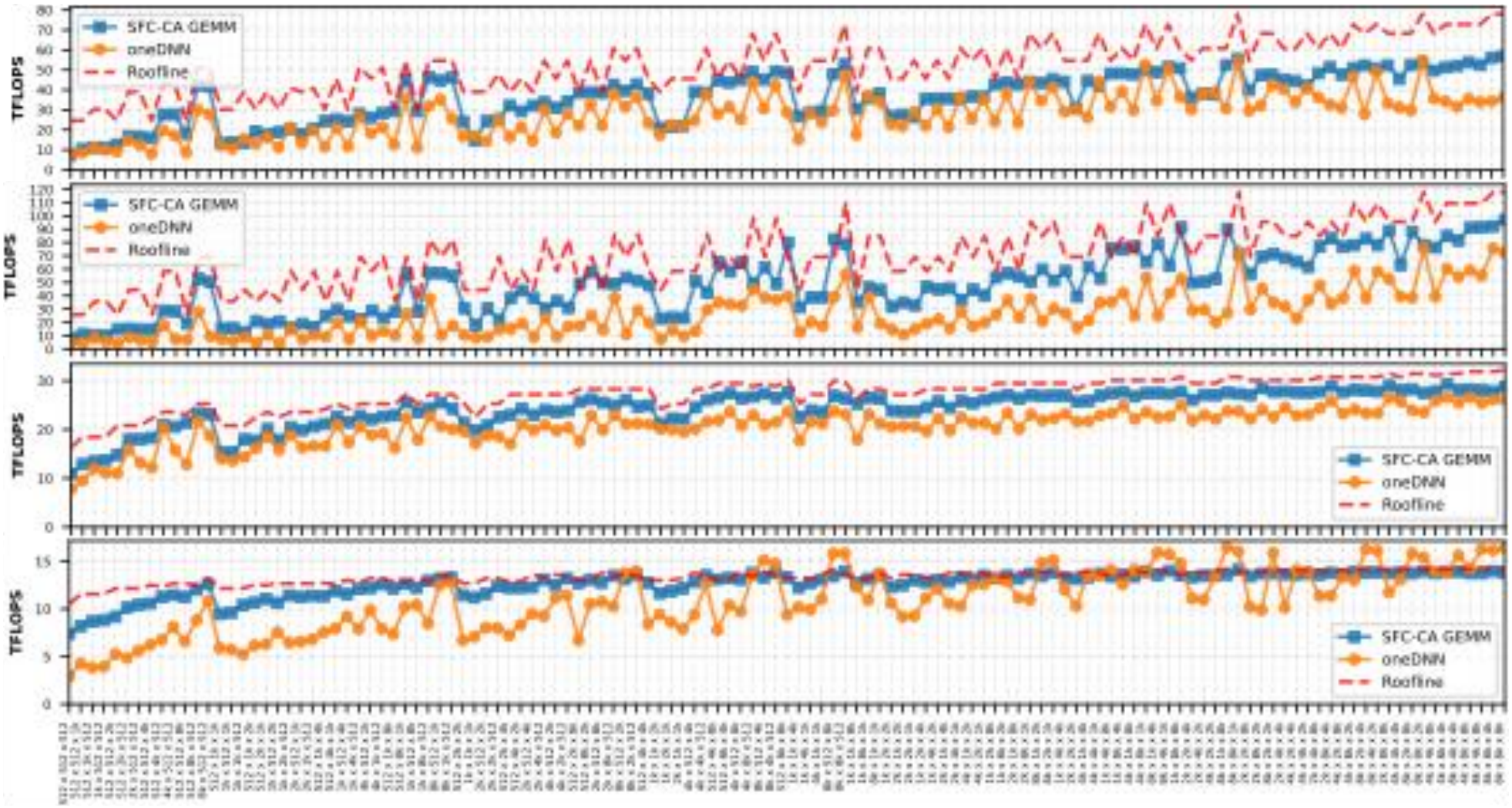
SFC-CA GEMM Algorithm

- By adjusting the K-layers parameter we ***implicitly*** get a class of Communication-Avoiding algorithms (2D, 2.5D and 3D GEMM)
- The algorithm attains the communication ***lower-bound*** → **provably minimizes data movement on the critical path for square/rectangular GEMM regimes!**





Performance portability across platforms



5th Gen Intel Xeon Emerald Rapids (64c): Up to 2.6x faster, 1.45x Weighted Harmonic Mean (WHM) speedup

6th Gen Intel Xeon Granite Rapids (128c): Up to 5.5x faster, 1.8x WHM speedup

5th Gen AMD EPYC (96c): Up to 1.7x faster, 1.15x WHM speedup

4th Gen AWS Graviton (96c): Up to 2.5x faster, on par WHM

Conclusions and Future Work

- SFC-CA GEMM: Space-Filling-Curve based GEMM implementing provably-optimal communication-avoiding matrix multiplication algorithms
- Simple implementation of ~30 C++ LOC
- Obviates the need for heuristics and auto-tuning on a per-platform and GEMM-shape basis
- Outperforms vendor-optimized libraries by up to ~1.8x (Weighted Harmonic Mean) across range of platforms (x86 and Arm).
- Performance closely tracks tight roofline model.
- Our work offers a robust Tensor Contraction method for Tensor Compilers
- Methodology extends to GPUs & higher-dimensional Tensor Contractions

Reffine: Efficient In-Memory Data Analytics through Sparse Compilation

Anand Jayarajan

NVIDIA, University of Toronto



OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

Reffine: A Unified Data Analytics Engine

SQL

GraphQL

APACHE Spark™

pandas

:

:



Field: The Data Abstraction in Reffine IR

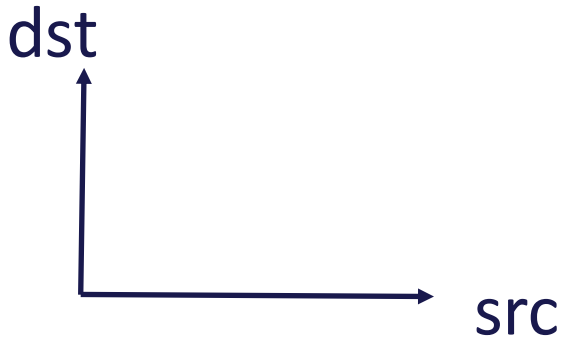
Data as a sparse, unbounded, multi-dimensional coordinate space

Streaming analytics



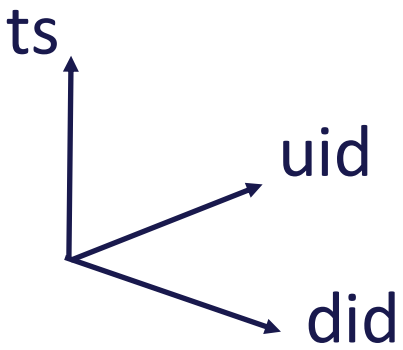
Data stream
(e.g., <ts> → temperature)

Graph analytics



Graphs
(e.g., <src, dst> → edge)

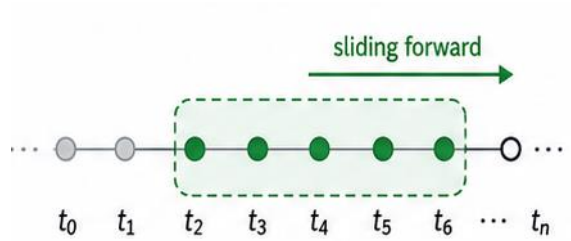
Relational analytics



Tables
(e.g., <uid, did, ts> → attributes)

Operator: The Compute Abstraction in Reffine IR

Streaming analytics

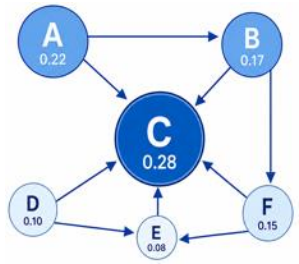



```
ma = ∀t:[Stock[t-50]≠φ |
         Stock[t]≠φ]
    {⨁(+, Stock[t-50:t])/ 50}

out = ∀t:[ ma[t]≠φ &
         Stock[t]≠φ ]
     { Stock[t] > ma[t] }
```

Stock price analysis

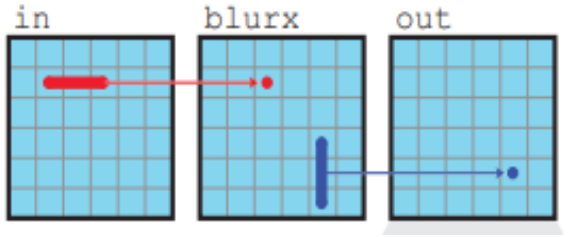
Graph analytics




```
M_hat = ∀x,y:[M[x,y]≠φ]
     {M[x,y]*0.85}
v = ∀x:[ M_hat[x]≠φ ] {
   dot = ∀y:[M_hat[x,y]≠φ &
          w[y]≠φ]{M[x,y]*w[y]}
   ⨁(+, dot) + (0.15/N)
}
```

Page ranking

Tensor programming




```
bx = ∀x,y: [x>=0 & y>=0 &
          x<512 & y<512]
     {img[x-1,y] + img[x,y]}

out = ∀x,y: [ bx[x,y-1]≠φ &
          bx[x,y]≠φ ]
     {bx[x,y-1] + bx[x,y]}
```

Image blurring

Optimizing Data Analytics Workloads

Unoptimized Reffine IR

```
v1 = ∀pk:[ Part[pk]≠φ &
    Part[pk].size>10 ]{}
v2 = ∀sk:[ Supp[sk]≠φ &
    Supp[sk].nation='US']{}

∀pk,sk:[v1[pk]≠φ &
        v2[sk]≠φ &
        PartSupp[pk,sk]≠φ ]
  { PartSupp[pk,sk].qty }
```

Fusion rule

if $F = \forall k : [\text{pred}]\ \{\text{val}\}$ then

$$F[k] \neq \phi \Rightarrow \text{pred}$$

$$F[k] \Rightarrow \text{val}$$

Optimized Reffine IR

```
~~v1 = ∀pk:[ Part[pk]≠φ &~~
~~    Part[pk].size>10 ]{}~~
v2 = ∀sk:[ Supp[sk]≠φ &
    Supp[sk].nation='US']{}

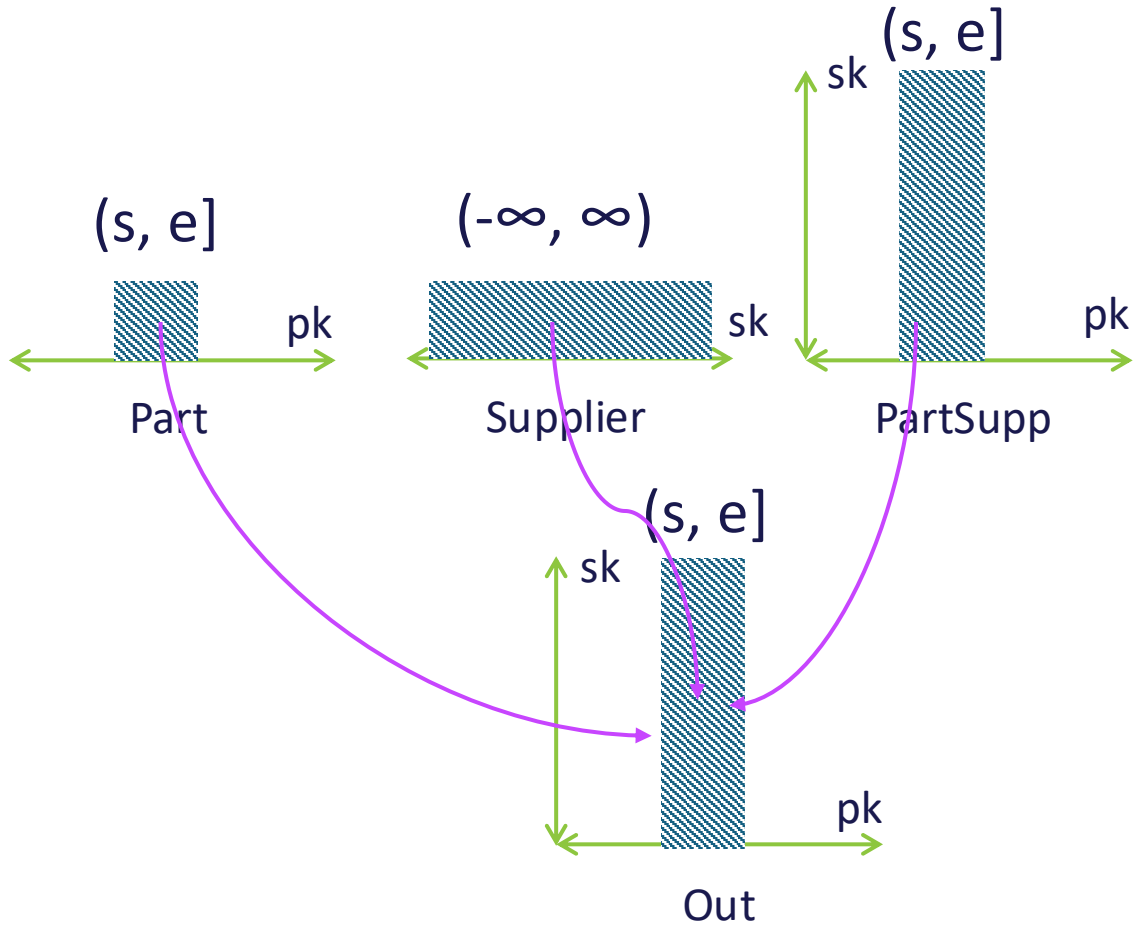
∀pk,sk:[Part[pk]≠φ &
        Part[pk].size>10 &
        v2[sk]≠φ &
        PartSupp[pk,sk]≠φ ]
  { PartSupp[pk,sk].qty }
```

Parallelizing Data Analytics Workloads

Reffine parallelize operators by injecting symbolic boundary conditions

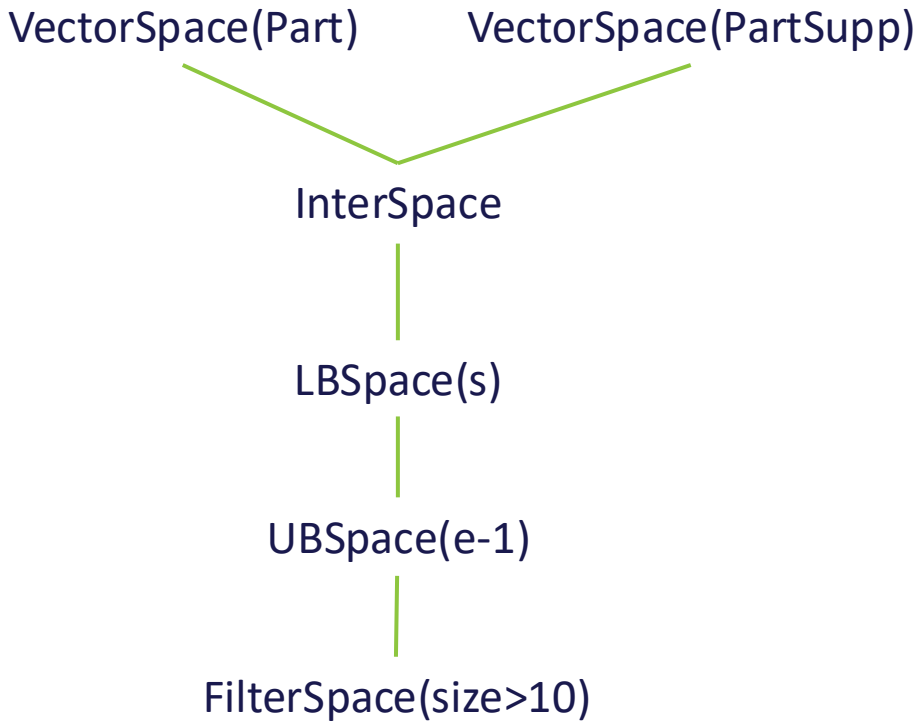
```
Out = ∀pk,sk:[
  Part[pk]≠φ &
  Part[pk].size>10 &
  s < pk & pk <= e &
  Supplier[sk]≠φ &
  Supplier[sk].nation='US' &
  PartSupp[pk,sk]≠φ ]
      { PartSupp[pk,sk].qty }
```

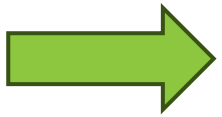
Parallelizing over *pk* dimension



Lowering Reffine IR to Loops: Sparse Compiler

$$\forall pk:[Part[pk]\neq\varphi \;\&\; PartSupp[pk]\neq\varphi \;\&\; s<=pk \;\&\; pk<e \;\&\; Part[pk].size>10]$$





```
idx = to_idx(lb())
while is_alive(idx) {
   iter = to_iter(idx)
   write_bit(out, cond(idx))
   write(out, (iter, value))
   idx = next(idx)
}
```

Partitioning Sparse Tensors for Load-Balanced Execution

Atharva Chougule

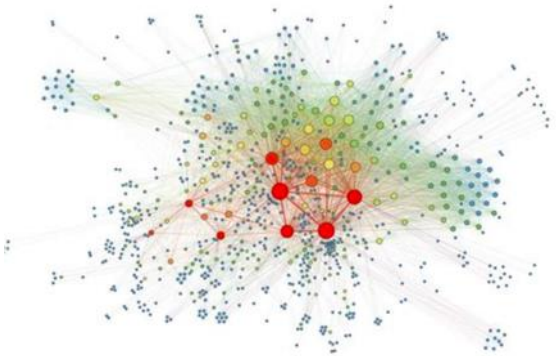
Stanford University



OCP FUTURE TECHNOLOGIES SYMPOSIUM

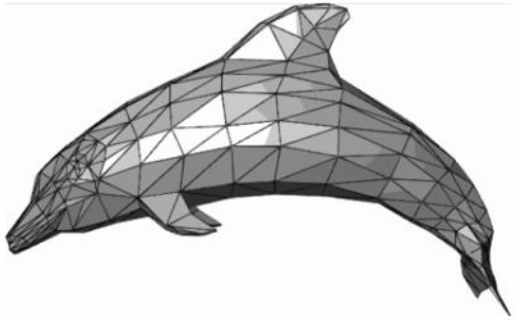
October 14, 2026
San Jose, California

Sparse Tensors Are Everywhere



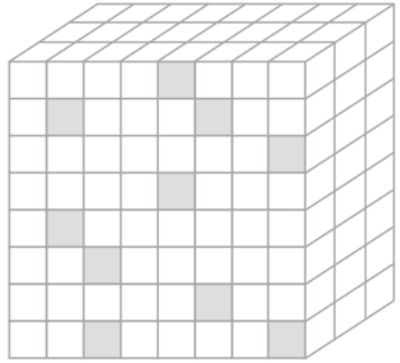
Social Networks

Caption



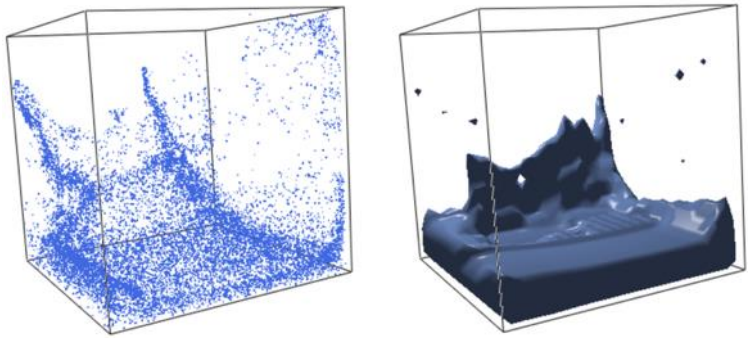
Conditional meshes

Caption



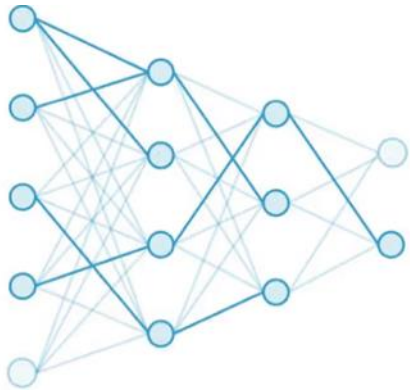
Recommendation tensors

Caption



Simulations

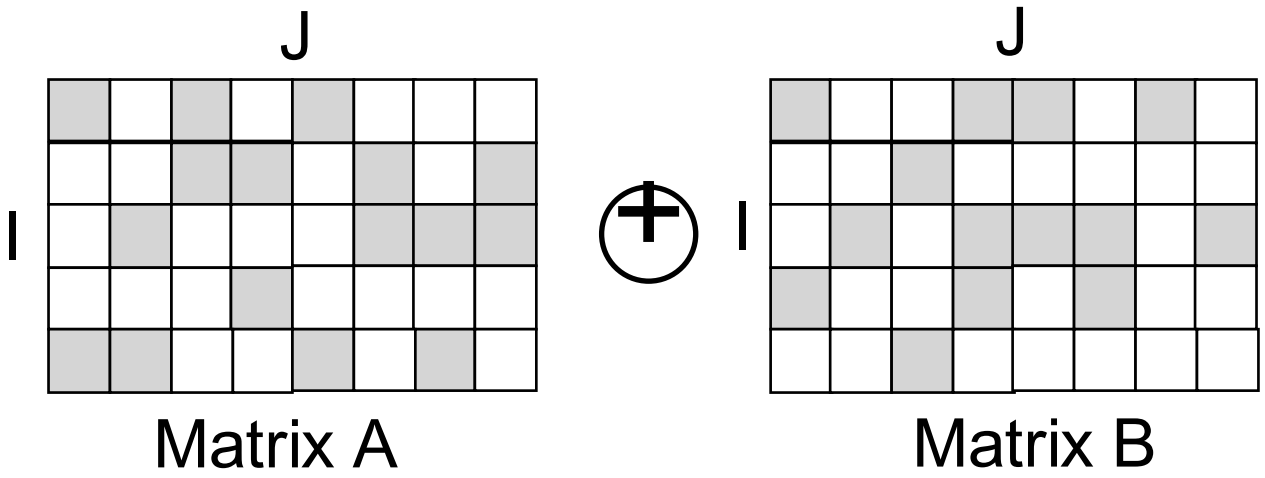
Caption



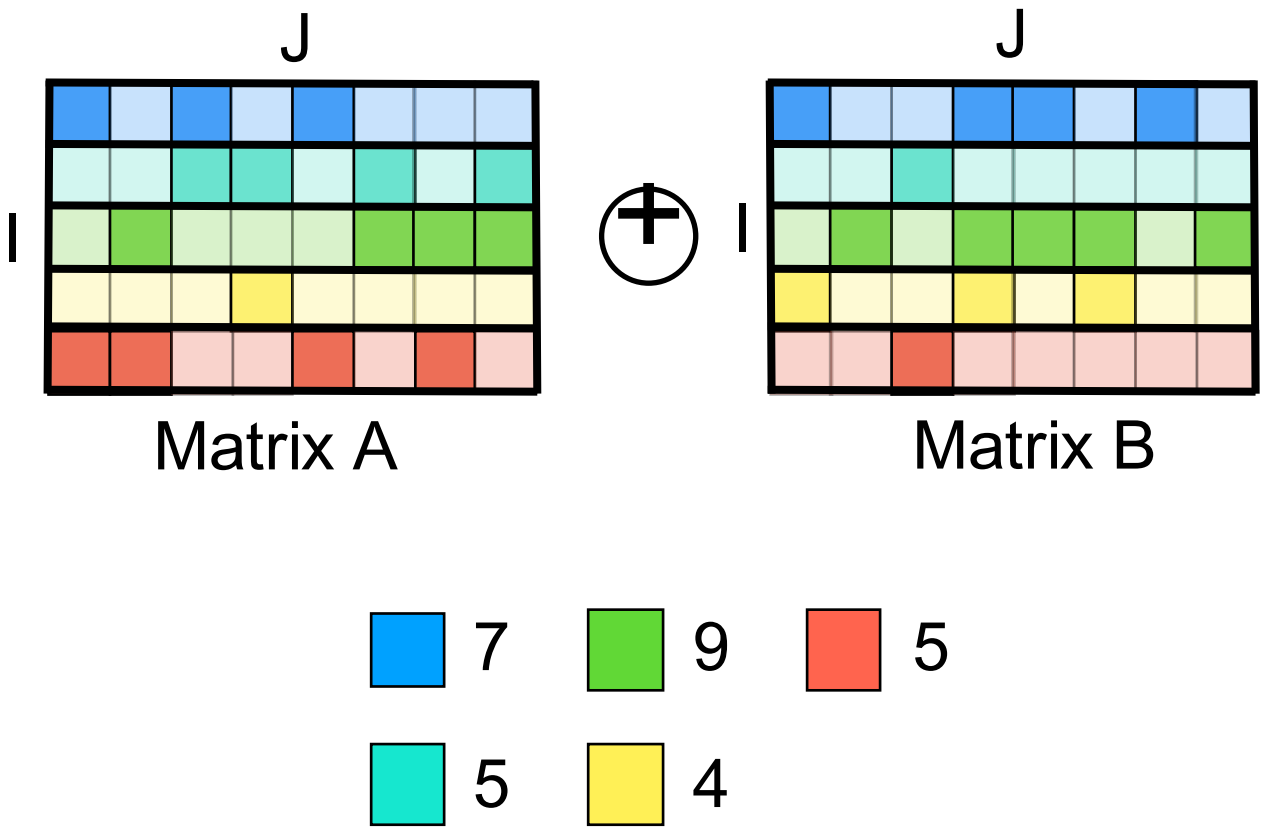
Sparse Neural Networks

Caption

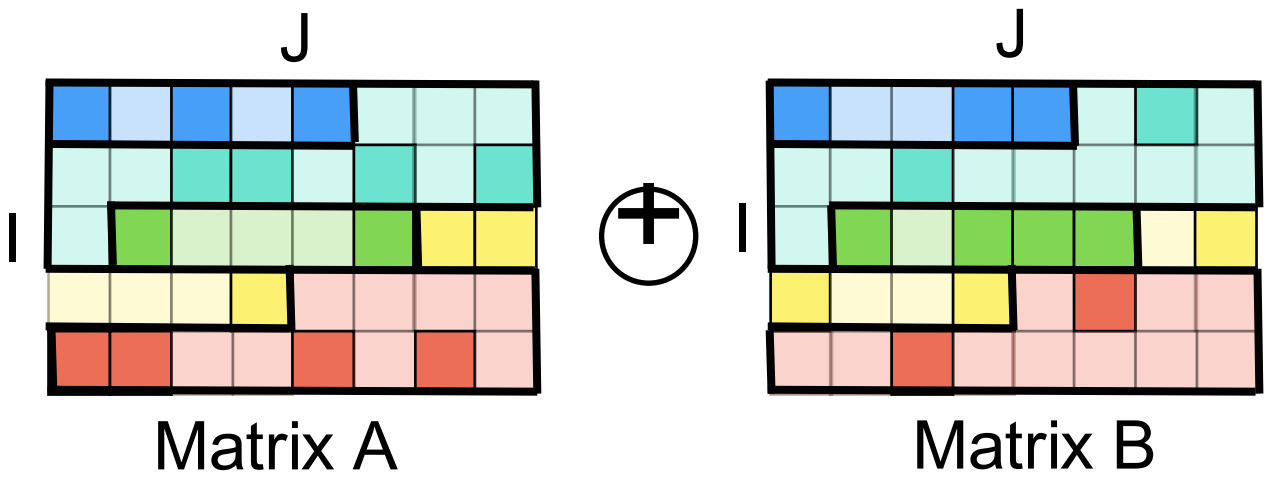
Partitioning Sparse Tensors for Load-Balancing is Hard



Partitioning Sparse Tensors for Load-Balancing is Hard



Partitioning Sparse Tensors for Load-Balancing is Hard



Load Balanced Partitioning is a function of how the data is distributed within the data structures!

Amount of Work in a Sparse Computation Depends on Multiple Factors

$$A = Bc + a \qquad a = Bc$$

$$A = B \odot C \qquad A = B + C \qquad a = \alpha Bc + \beta a$$

$$A = BCd \qquad A = \alpha B \qquad A = 0 \qquad A = BC$$

$$a = b \odot c \qquad A = B \odot (CD)$$

$$A_{ij} = \sum_{kl} B_{ikl} C_{lj} D_{kj} \qquad A = B^T \qquad a = B^T Bc$$

$$A_{ik} = \sum_{j} B_{ijk} c_j \qquad A_{kj} = \sum_{il} B_{ikl} C_{lj} D_{ij}$$

$$A_{ijk} = \sum_{l} B_{ikl} C_{lj} \qquad A_{ij} = (\sum_{k} B_{ijk} C_{ijk}) + D_{ij}$$

$$\mathcal{C} = \sum_{ijkl} M_{ij} P_{jk} \overline{M_{lk}}\, \overline{P_{il}} \qquad \tau = \sum_{i} z_i (\sum_{j} z_j \theta_{ij})(\sum_{k} z_k \theta_{ik})$$

$$a = \sum_{ijklmnop} M_{ij} P_{jk} M_{kl} P_{lm} \overline{M_{nm}} P_{no} \overline{M_{po}}\, \overline{P_{ip}}$$

Caption

Tensor Expression

Dense Matrix CSR BCSR

COO DCSR ELLPACK CSB

DIA Blocked COO CSC

Blocked DIA DCSC

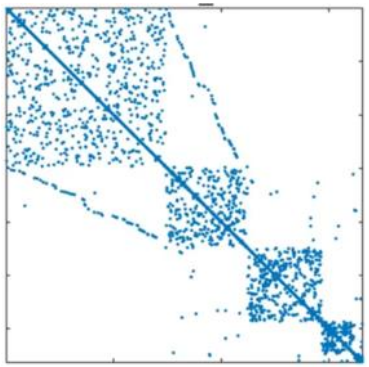
Sparse vector Hash Maps

CSF Dense Tensors

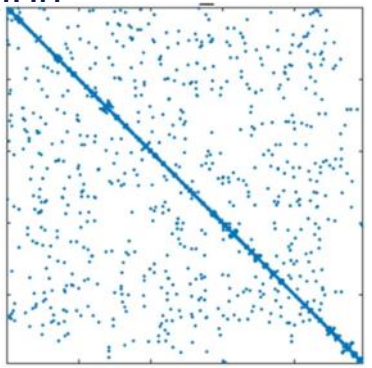
Blocked Tensors

Caption

Sparse Data Formats



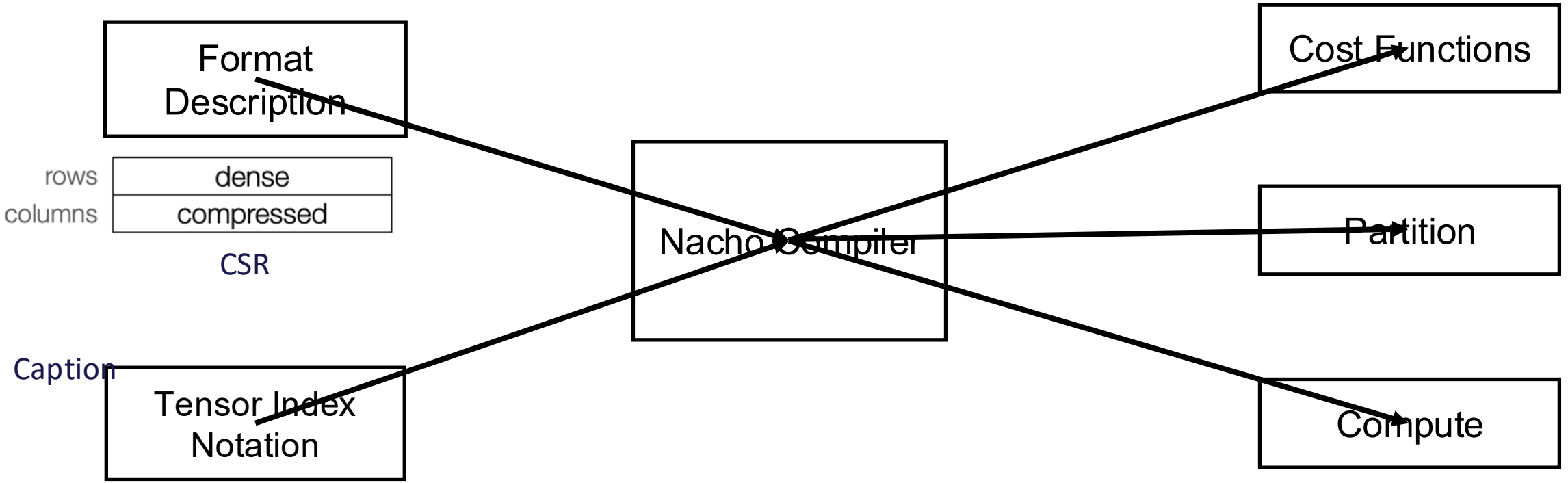
Caption



Caption

Data Distribution

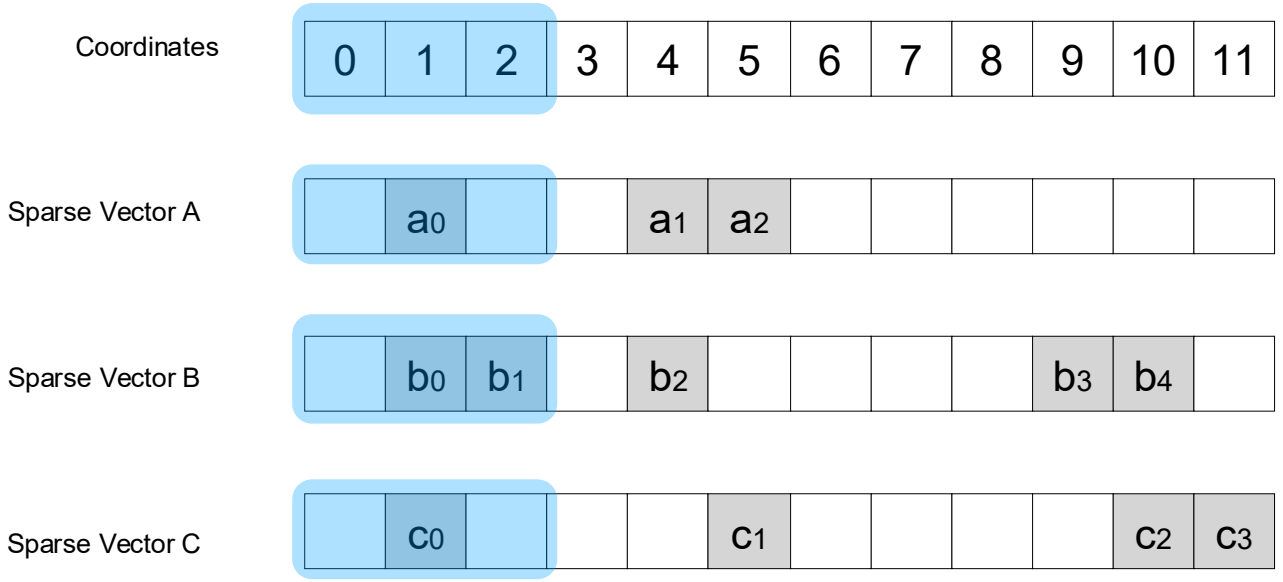
Nacho: A Load-Balancing Sparse Tensor Compiler



* Chougule, A., Root, A.J. "Partitioning Unstructured Sparse Tensor Algebra for Load-Balanced Parallel Execution." ASPLOS '27.

$C_{ij} = A_{ij} + B_{ij}$

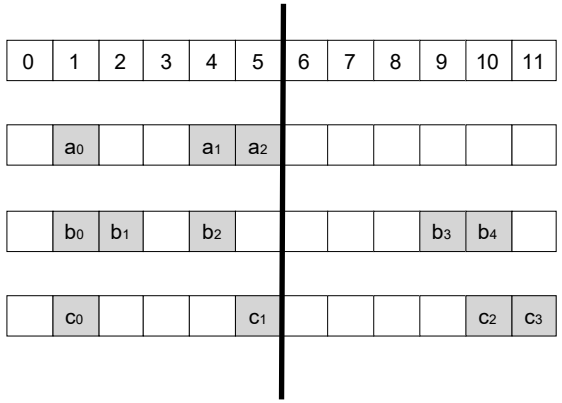
Partitioning Sparse Tensors is a search problem



$$C(3) = 4$$

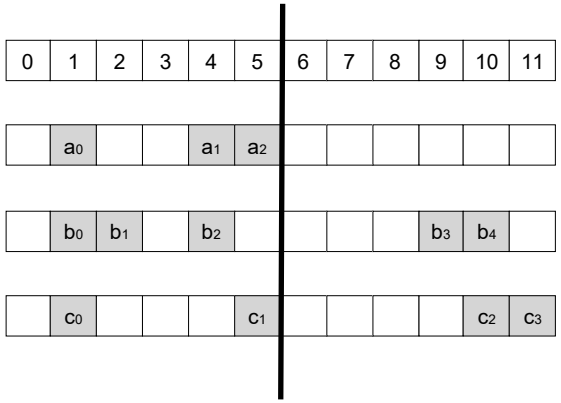
Let C(x) = Number of non-zeros contained within the range [0, x)

Partitioning Sparse Tensors is a search problem



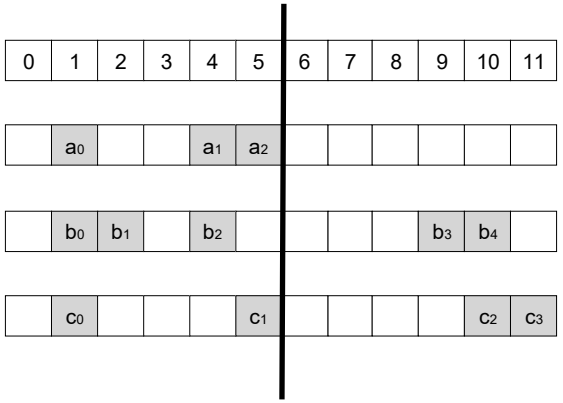
$C(6) = 8$

Search for $C(x) = 3$



$C(6) = 8$

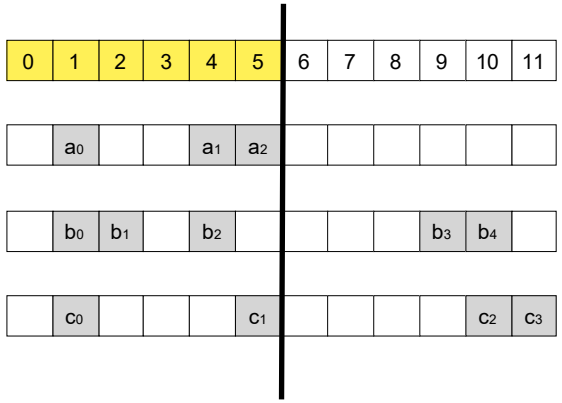
Search for $C(x) = 6$



$C(6) = 8$

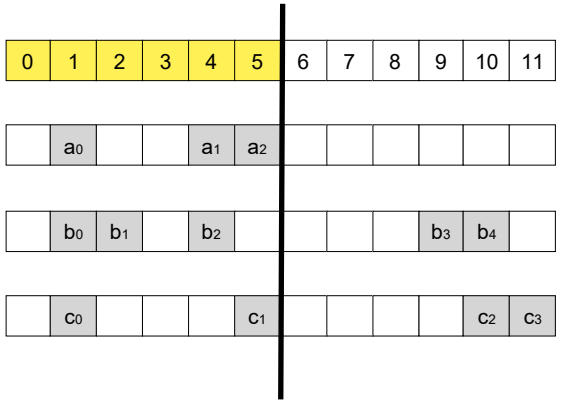
Search for $C(x) = 9$

Partitioning Sparse Tensors is a search problem



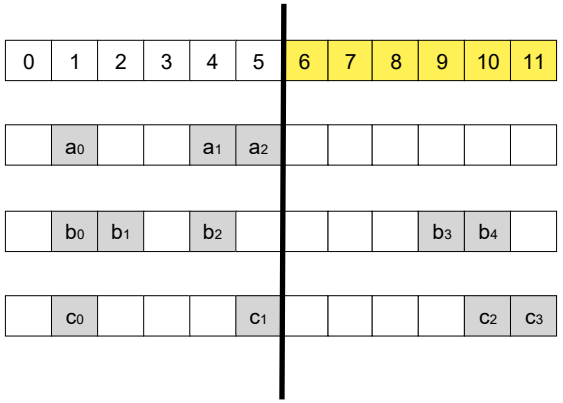
$C(6) = 8$

Search for $C(x) = 3$



$C(6) = 8$

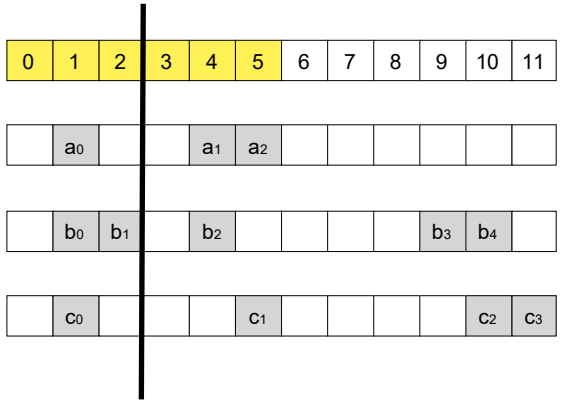
Search for $C(x) = 6$



$C(6) = 8$

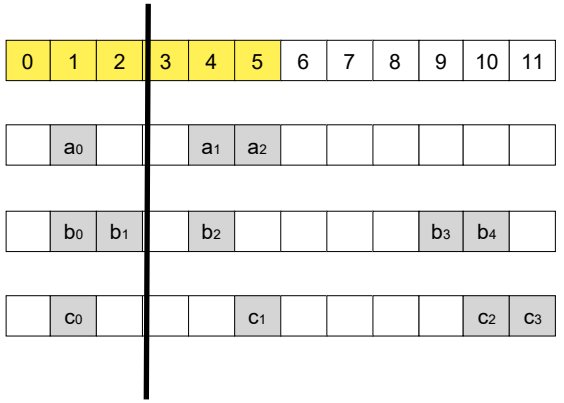
Search for $C(x) = 9$

Partitioning Sparse Tensors is a search problem



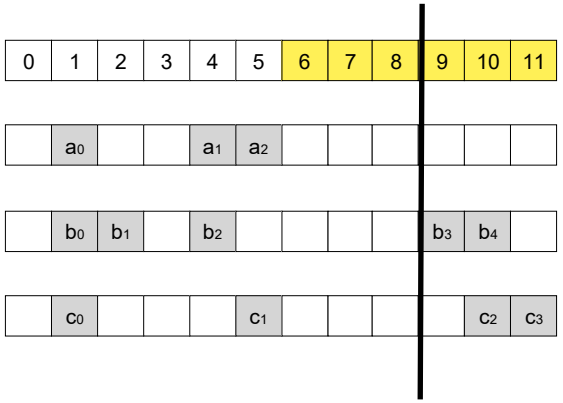
$C(3) = 4$

Search for $C(x) = 3$



$C(3) = 4$

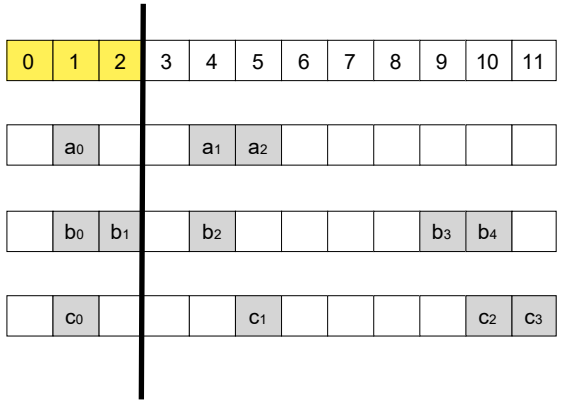
Search for $C(x) = 6$



$C(9) = 8$

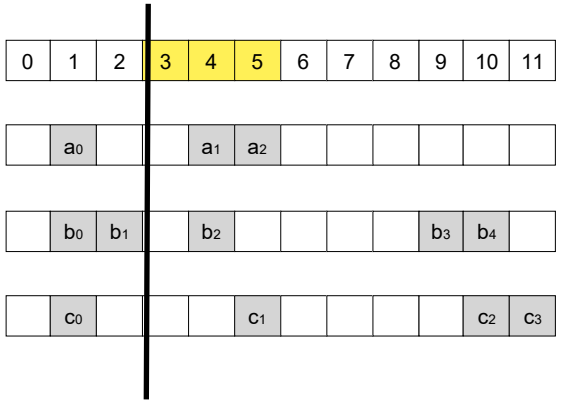
Search for $C(x) = 9$

Partitioning Sparse Tensors is a search problem



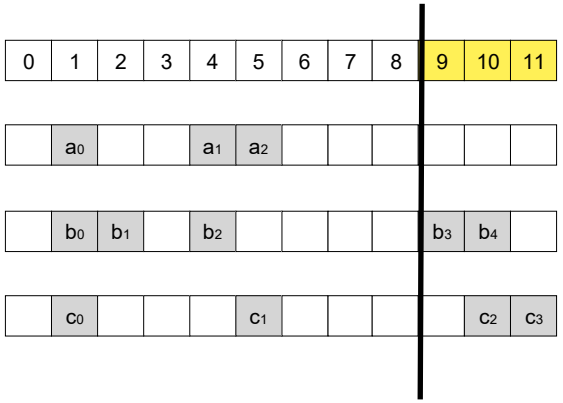
$C(3) = 4$

Search for $C(x) = 3$



$C(3) = 4$

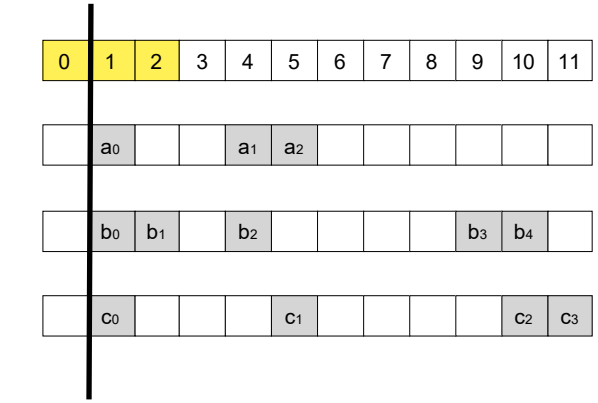
Search for $C(x) = 6$



$C(9) = 8$

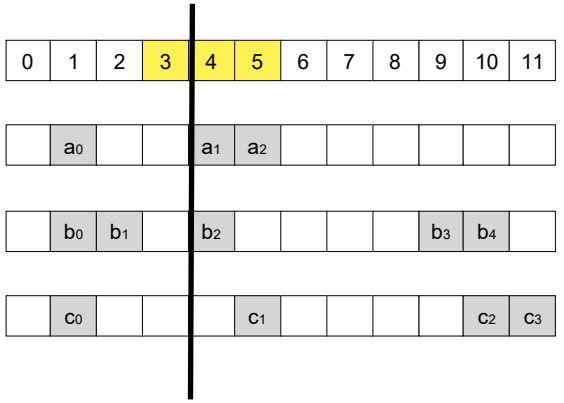
Search for $C(x) = 9$

Partitioning Sparse Tensors is a search problem



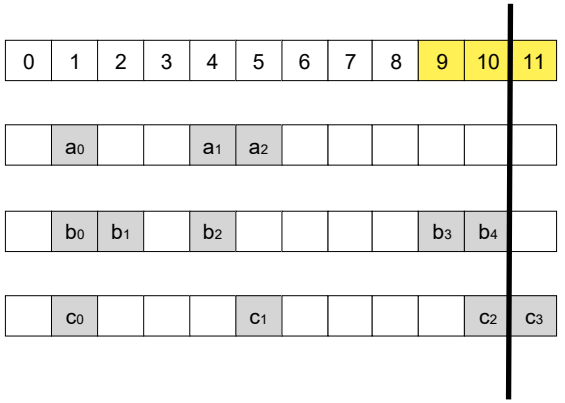
$C(1) = 0$

Search for $C(x) = 3$



$C(4) = 4$

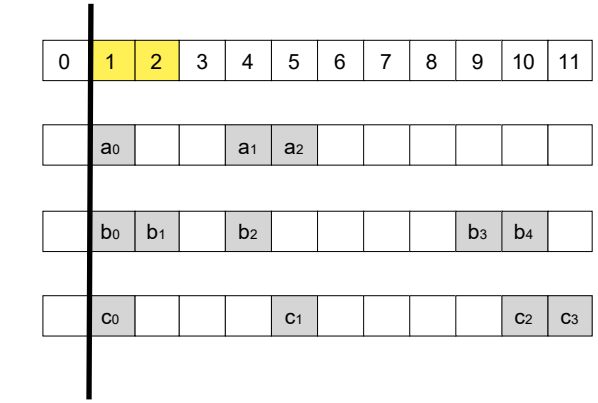
Search for $C(x) = 6$



$C(11) = 11$

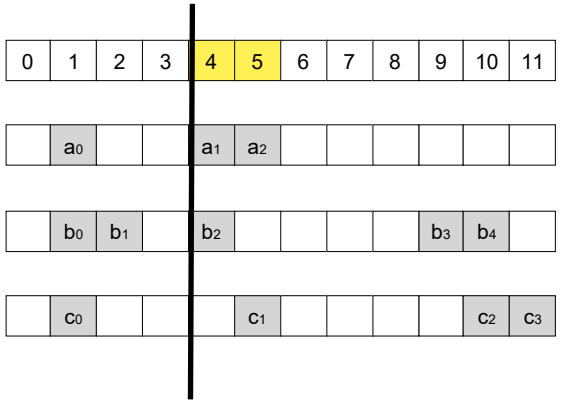
Search for $C(x) = 9$

Partitioning Sparse Tensors is a search problem



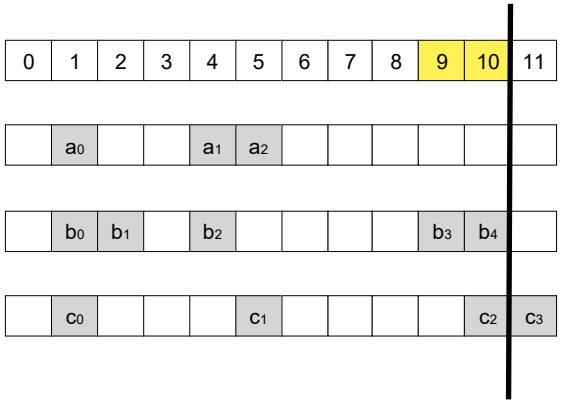
$C(1) = 0$

Search for $C(x) = 3$



$C(4) = 4$

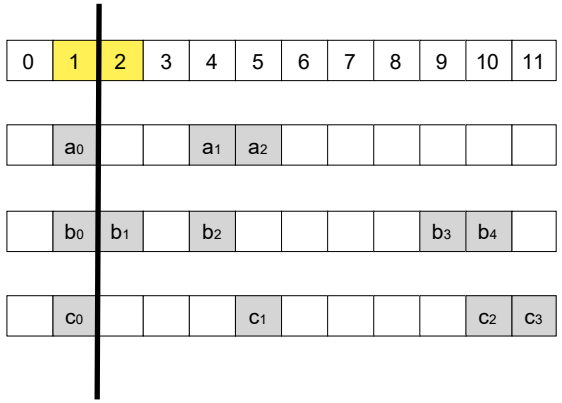
Search for $C(x) = 6$



$C(11) = 11$

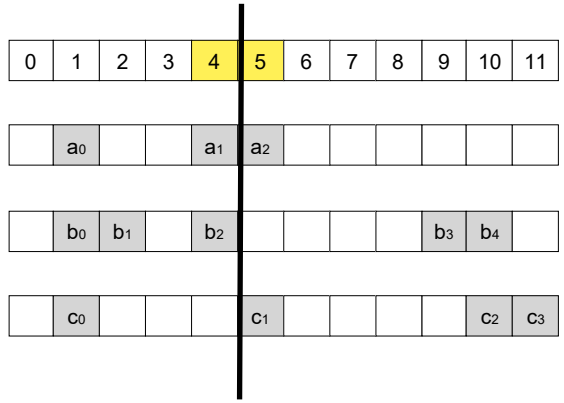
Search for $C(x) = 9$

Partitioning Sparse Tensors is a search problem



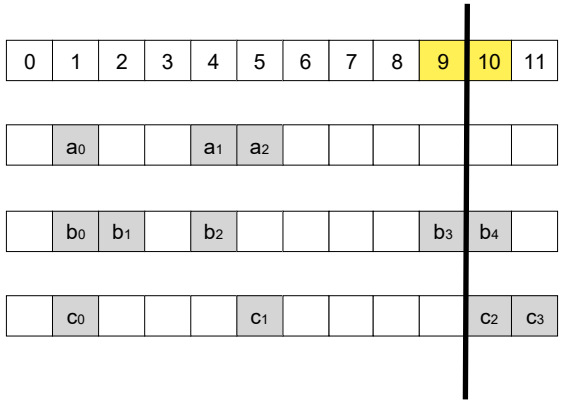
$C(2) = 3$

Search for $C(x) = 3$



$C(5) = 6$

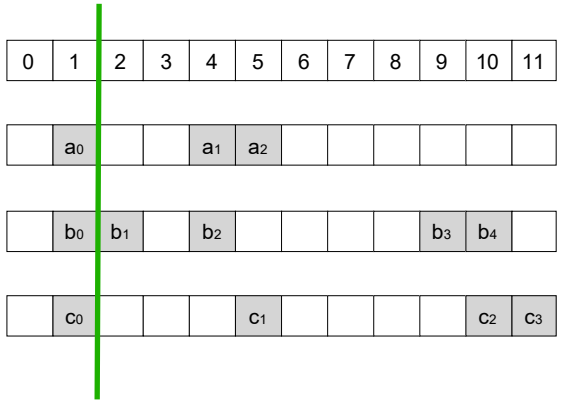
Search for $C(x) = 6$



$C(10) = 9$

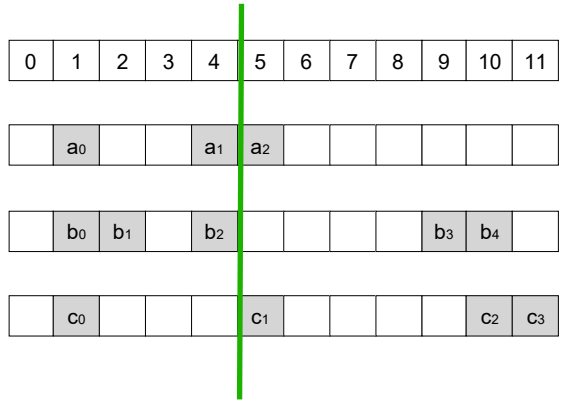
Search for $C(x) = 9$

Partitioning Sparse Tensors is a search problem



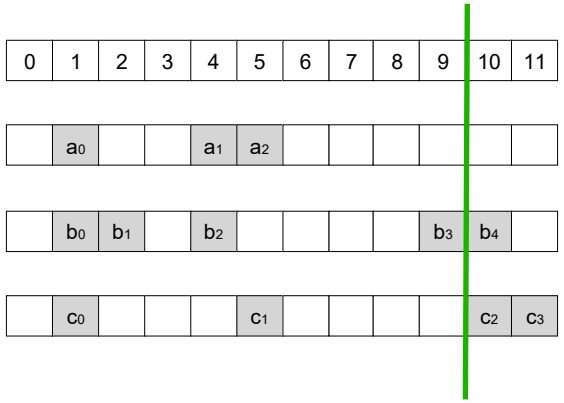
$C(2) = 3$

Search for $C(x) = 3$



$C(5) = 6$

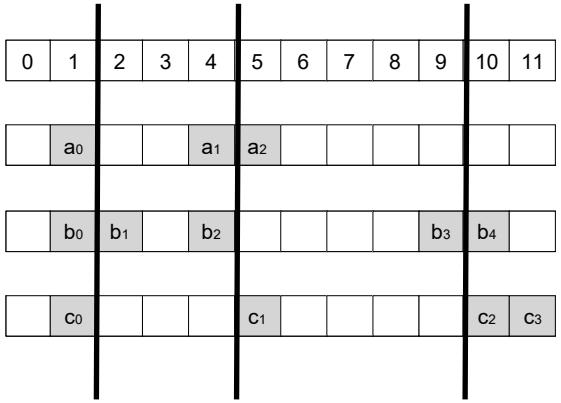
Search for $C(x) = 6$



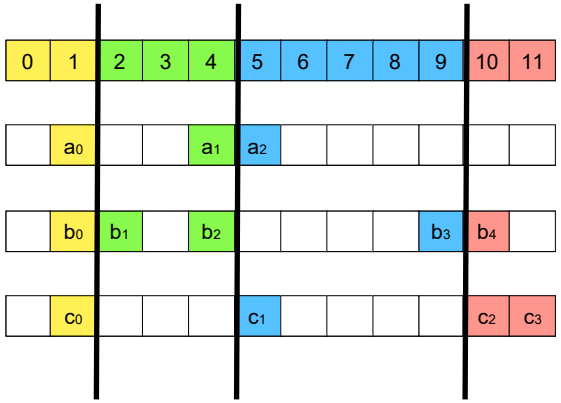
$C(10) = 9$

Search for $C(x) = 9$

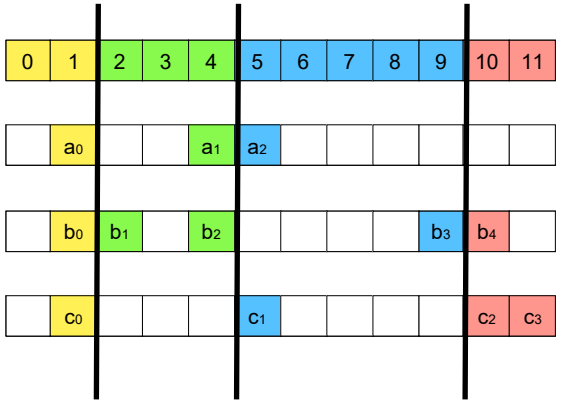
Partitioning Sparse Tensors is a search problem



Partitioning Sparse Tensors is a search problem



Partitioning Sparse Tensors is a search problem

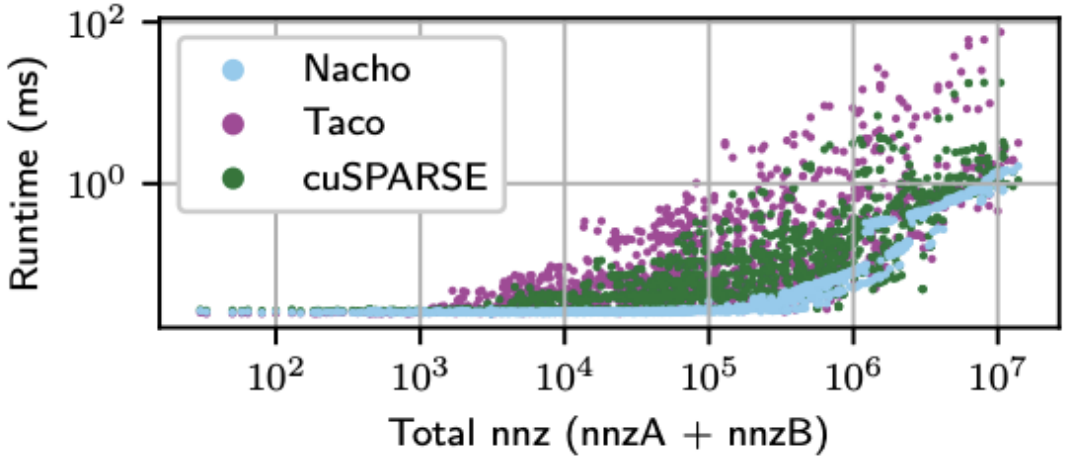


Extending to any n-dimensional tensors

- Per-dimension Cost function
- Per-dimension Hierarchical Search

* Chougule, A., Root, A.J. "Partitioning Unstructured Sparse Tensor Algebra for Load-Balanced Parallel Execution." ASPLOS '27.

Load-Balanced Partitioning Leads to Better Performance



Caption

geo-mean speedup 2.4x (Taco)
and 1.8x (cuSPARSE)

* Chougule, A., Root, A.J. "Partitioning Unstructured Sparse Tensor Algebra for Load-Balanced Parallel Execution." ASPLOS '27.

OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California



Additional Questions

- Open Questions
  - Can well-specified partitioning operators eliminate the burden of specifying how partial results should be *gather*ed and *reduce*d?
  - To what extent do we need to investigate partitioning operators on open formats for graphs and tabular data instead of cross-compiling graph algos and analytical queries while also converting data into linear algebra types?
  - Aren't open formats for compute inevitable?
- Additional Operator and Types?
  - Should we look to standardize any Randomized Operators for Parallel DFS Numbering of graphs, multi-objective graph partitioning such as NSGA-II?
  - Extend Arrow open formats for data and compute to include emerging Compressed Embedding and Attention vector types (e.g. in Random Hadamard Rotations of TurboQuant) so that Nearest Neighbor searches occur in quantized domain?